

已知错误理由条件下引导 LLM Planner 改写 Action 的提示干预研究

Executive Summary

本文讨论的是一个比“风险判定”更窄、也更可实验化的问题：**在 verifier 已经指出某个 planner/agent/LLM 的候选 action 有问题，并给出理由之后，如何仅通过 prompt 或外部信息注入，最大化模型改写内部决策、转向正确 action 的概率。**现有文献整体上支持一个清晰结论：在这一设定下，最可靠的信号不是泛化的“请再想一想”，而是**外部、过程级、动作局部化的反馈**。Kamoi 等对自我纠错文献的综述明确指出，现有工作里真正稳定有效的自我纠错多依赖**可靠的外部反馈**；Lightman 等显示过程监督优于仅看最终结果的 outcome supervision；Zhang 等进一步表明，哪怕是较小模型，只要接入**强 verifier**，自我修正能力也会显著提升。

1

从“如何写 prompt”看，文献最支持的不是把理由写得更长，而是把理由写得**更局部、更对比、更可执行**。约束分解与 critique-refine 类方法表明，把错误 action 绑定到它违反的具体约束，能显著提升跟随正确约束的概率；对比式 in-context 学习与 contrastive prompting 则表明，**同时展示“错误为什么错”与“正确为什么更好”**，比只说“请修正”更容易改变模型的决策边界。综合这些结果，一个高性价比的默认方案是：“**错误 action + 局部错误点 + 结构化约束/前提 + 自然语言理由 + 一个最小替代 action 要求**”，而不是笼统反思。

2

相反，若只靠长上下文堆叠历史、或用角色/身份设定来“激发正确性”，证据要弱得多。长上下文研究显示，相关信息被埋在中间位置时性能会显著下降；few-shot 研究还表明示例顺序高度敏感。角色 prompt 在客观任务上通常不能稳定提升正确率，且效果常带随机性或与 persona 选择偶然相关。也就是说，如果你已经知道“哪里错、为什么错”，最有效的操作往往不是“再多给一点历史”，而是**把关键信息压缩后放到当前 action 决策点前**。

3

关于置信度，结论也相当一致：**它适合作为门控变量，不适合作为单独的纠错内容**。一方面，模型在合适格式下有一定自知之明，能表达自己“知道/不知道”的程度；另一方面，口头化置信度高度依赖提问方式，且经常过度自信。因此，把 verifier 的置信度或严重性分数用于“是否触发修正 / 是否允许回溯 / 允许几轮修正”的 gating 是有价值的，但单独给一个分数，通常不足以改变 action。

4

若把这些证据转成一条简明判断，则可以概括为：**在已知错误与理由的前提下，成功改变 agent 内部决策的关键不在于增加提示语总量，而在于提高理由的诊断性、局部性、对比性和控制强度**；对这个问题，最应避免的是无约束的多轮自反思、冗长历史堆叠、以及把“角色扮演”误当作可靠的纠错手段。

5

可直接落地的结论可以压缩为四点。

- 最优先的干预对象应是**当前错误 action 的局部决策点**，而非全局摘要。
- 最有效的理由编码通常是**自然语言解释 + 结构化标签/约束 + 明确禁止重复原 action**的混合式编码。
- 若理由较复杂，应优先使用**对比式或反事实式重写**，把“继续原 action 会怎样”和“改成什么 action 更满足约束”写清楚。
- 自我反思最多应作为**被外部理由触发的一次或两次有限修正**，而不应默认无限循环。

理论机制与问题结构

在你的设定中，问题可以形式化为：给定任务输入 x 、历史 h_t 、模型当前候选 action a_t^{bad} ，以及 verifier 返回的错误理由 r_t ，寻找一种上下文干预 $\pi(r_t)$ ，使修正后 action a_t' 变为正确 action a_t^{corr} 的条件概率最大化，同时控制 token 预算与回合数。这里的关键不是重新训练模型，而是利用**上下文条件化**改变下一个 action token 的分布。Kamoi 等的综述说明，这一类 test-time self-correction 若没有可靠外部反馈，通常并不稳定；而在有强 verifier 的前提下，问题从“内生自纠”变成了“外部反馈引导下的条件决策重排”，难度会明显下降。

10

第一类机制是**提示语语义与指令遵从性**。InstructGPT 与 Constitutional AI 都表明，经过 instruction tuning / RLHF / RLAIF 的模型，会对自然语言中的任务要求、规则、批评与修订请求产生可利用的行为偏置；Zero-shot CoT 和 APE 又进一步说明，哪怕是很短的提示语变化，也能显著改变模型的推理轨迹与任务表现。对你的问题而言，这意味着 verifier 的理由若被写成**明确、权威、局部的“决策约束”**，通常比写成松散评论更能推动 action 改写。

11

第二类机制是**上下文利用、在上下文学习与元学习**。Min 等发现，demonstration 的作用并不只来自“标签真值”，还来自标签空间、输入分布与输出格式本身；Liu 等显示，语义相近的示例优于随机示例；Lu 等则表明 few-shot prompt 的示例顺序高度敏感；von Oswald 等提出 ICL 与梯度下降式元学习存在机制上的联系，而 Shen 等提醒这种等价在真正的预训练模型里仍是开放问题。对本题的直接启发是：**理由编码的“格式”和“位置”本身就是干预变量**。如果 verifier 信息以稳定 schema、语义相似示例、并贴近 action 生成位置出现，模型更容易把它当成当前局部任务的一部分，而不是背景噪声。

12

第三类机制是**对比、反事实、过程监督与反思**。Contrastive in-context learning 与 contrastive prompting 表明，显式展示正反例或要求模型同时给出“正确与错误”的候选，能强化区分边界；Self-Refine 与 Reflexion 说明语言化 critique 可以推动下一轮更好的生成；CoVe 则把“草稿—验证问题—独立回答—修正”做成一种系统化流程。与此同时，Huang 等与 Kamoi 等都强调：**没有外部反馈时，纯自我纠错在推理任务上常常不可靠，甚至会退化**。因此，本题最稳妥的路径不是“相信模型会自发意识到自己错了”，而是用 verifier 的理由把错误定位成一个过程级、可反驳、可替代的局部决策。

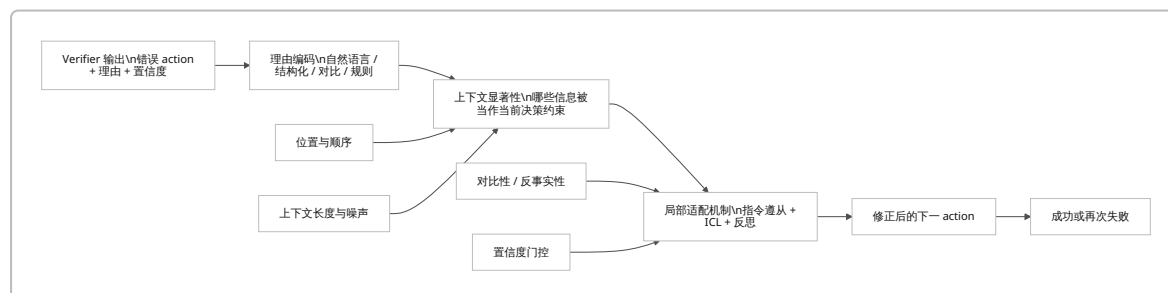
13

第四类机制是**置信度与校准**。Kadavath 等显示，大模型在某些设定下能较好判断自己知道什么；Xiong 等指出 verbalized confidence 常常过度自信；Yang 等进一步表明这种口头置信度对 prompt 形式非常敏感；Li 等的“Confidence Matters”则说明在自我纠错中忽略置信度会导致过度自我批判。对 action 修正任务，这意味着置信度最好的用途往往不是“说服模型”，而是**控制修正策略**：例如高置信 verifier 触发“硬回退/禁止重复 action”，低置信 verifier 只触发“软重写”。

4

下面这个因果图概括了该问题中最重要的作用链条：外部理由先决定上下文中哪些特征变得显著，随后通过指令遵从、ICL 与局部反思机制，改变 action 的条件分布；而位置、顺序、对比性与置信度决定这种改变是否会成功。综合 Lost in the Middle、顺序敏感性研究与对比式提示结果，这里真正关键的是把**“错因”放到离 action 最近的位置，并使其具有可执行的替代含义**。

14



Prompt 干预范式比较

现有 prompting 文献已经形成较完整的技术谱系，但在你的设定下，真正重要的不是“哪种 prompt 最流行”，而是“哪种 prompt 最能在**已知错因**后改变 action”。下表按照这一标准比较主要范式；表中判断综合了 prompting 综述、原始 prompting 论文以及自我纠错/约束跟随文献。¹⁵

范式	原理	优缺点	适用场景	可测指标
短提示	用极短 cue 直接触发“重新审视当前 action”的推理模式，例如要求说明为何该 action 错、并给出最小修正；零样本短语本身就可能显著改变推理路径。 ¹⁶	优点是便宜、对照干净、复现实验容易；缺点是承载复杂错因的能力弱，对高置信错误或多约束冲突往往不够。 ¹⁷	错误局部、可一句话描述、动作空间较小。	单次修正成功率、额外 token、首轮改写率。
长上下文	把 verifier 理由、历史轨迹、环境状态一并放入上下文，让模型自行重加权。Generated Knowledge Prompting 表明额外知识能帮助推理，但长上下文利用并不鲁棒。 ¹⁸	优点是信息最全；缺点是“中间丢失”、噪声干扰、位置敏感严重。 ¹⁹	理由需要依赖较长任务历史或环境证据。	成功率、对理由位置的敏感度、token 成本、信息效率。
反事实提示	把理由写成“若继续执行该 action，会违反什么；若改成另一 action，哪些约束会恢复满足”的最小世界变动。该思路在特定推理一致性任务上有效，但普适证据仍较窄。 ²⁰	优点是直接作用于“行动后果”层；缺点是对模型的 counterfactual 能力要求较高，复杂任务中可能引入额外幻觉。 ²¹	状态转移明确、前提/后果可描述、计划修补。	纠错成功率、状态一致性、后续违约率。
对比示例	同时给出“错误 action/答案”与“更优 action/答案”，或要求模型先给出一错一对再做选择；对比式 ICL 与 contrastive prompting 都支持这种边界强化效应。 ²²	优点是最容易形成可观察的决策边界；缺点是需要负例或可构造的反例，提示稍长。 ²³	错误模式稳定、可收集正反例、理由较语义化。	成功率、负例覆盖率、对新错误类型的泛化。
反思或自我纠错触发	先让模型批判当前输出，再让其自我修正；Self-Refine、Reflexion、CoVe 都属于这一类。 ²⁴	优点是实现简单、与现有 agent 框架兼容；缺点是若没有外部理由或强 verifier，推理任务中常不稳定甚至退化。 ²⁵	已有外部 critique、希望最小改动现有系统。	每轮增益、循环次数、退化率、首轮修正成功率。

范式	原理	优缺点	适用场景	可测指标
约束式提示	把 verifier 理由分解为显式约束、前提、禁止项、格式要求，再要求模型在这些约束下重写 action；DeCRIM 对多约束指令跟随尤其有效。 ²⁶	优点是控制力强、便于自动评估、最适合“已知哪里违反了规则”；缺点是在隐性语义错误上可能过硬、表达不够细腻。 ²⁷	多约束任务、格式约束、工具调用前提检查。	约束级准确率、指令级成功率、误杀率。
分步提示加回溯	让模型显式分解子目标，并在 verifier 指出某步错误后回溯到上一步或上一个子目标；Least-to-Most、ToT、ReAct 都体现了这种思想。 ²⁸	优点是适合长任务与早期错误修复；缺点是 token 成本高，若没有门控容易形成循环。 ²⁹	多步规划、Web/embodied agent、代码修补。	回溯深度、平均步数、任务成功率、循环死锁率。
元提示	用 prompt 指导模型如何读 verifier 理由、如何优先修改、如何在失败案例上自我生成更优提示；APE 与 RePrompt 属于此类。 ³⁰	优点是适合离线寻找更优提示模板；缺点是不一定比直接的局部纠错更强，且容易走向 prompt-overfitting。 ³¹	做论文、做系统性 ablation、固定任务分布。	模板泛化、跨任务迁移、对 paraphrase 的稳定性。
角色或身份设定	通过“你是严谨审稿人/规划师/检验员”等身份来诱导更谨慎的决策。	现有强证据显示 persona 对客观正确率 通常不稳定 ，很多情况下不优于无 persona；不同 persona 还会随机影响预测。 ³²	更适合风格、语气、模拟任务，不适合作为主要纠错机制。	正确率变化、方差、对 persona 选择的敏感度。
外部工具调用提示	把 verifier、检索器、计算器、环境 API 写成“先行动作”，令模型在修正前先查证或先验证。ReAct 与 Toolformer 都表明工具能改变后续 token 分布。 ³³	优点是把“理由”变成外部证据而非纯语词；缺点是系统依赖增加，且面临 prompt injection 与工具误调度。 ³⁴	事实型错误、算术、Web 任务、环境交互任务。	工具调用率、调用后成功率、额外延迟、注入鲁棒性。

就你的问题而言，若追求**最小设计、最大可发表性**，优先级大致可以排成：**约束式提示** ≈ **对比式提示** ≈ **外部理由触发的单轮反思**，其次是**限次回溯**，再次是**长上下文**，最后才是**角色设定**。这个排序并不是说后者没有用，而是说在“错因已知”的设定下，最有效的信号应当直接作用于当前错误 action 的判别边界，而不是通过风格或大段背景间接影响模型。³⁵

如何编码理由以促成行为改变

如果把 prompt intervention 看成“改变模型对下一 action 的局部条件化”，那么**理由编码方式**本身就是核心自变量。综合现有证据，我更推荐把理由看成一个由三部分组成的对象：**局部错误定位、约束/前提标签、可执行替代方向**。原因是，纯自然语言解释虽然表达力强，但未必给出可操作的决策边界；纯结构化标签虽然清

晰，却容易缺少语义诊断。DeCRIM、IFEval 和 demonstration 研究共同暗示：在上下文里，**稳定 schema 与语义解释的混合**最容易触发改写。³⁶

自然语言理由最适合语义性的、难以离散枚举的错误。例如“该 action 过早调用工具，因为必要参数还未从页面读取”这类理由很难只用标签表达。Self-Refine、Reflexion、Constitutional AI 都表明，语言化 critique、原则、修订建议本身就能改变下一轮生成。缺点在于自然语言容易冗长、模糊，尤其当理由中同时混入多个错误点时，模型未必知道应该修改哪个 decision variable。³⁷

结构化标签适合把理由变成模型更容易“对齐”的局部控制量，例如：`error_step=t-1`、`violation=missing_precondition`、`forbid=repeat_action`、`severity=high`。Min 等关于 label space/format 的发现，以及 DeCRIM/IFEval 关于可验证约束的结果，都支持这种做法：格式稳定、字段固定、约束可枚举时，模型更容易把反馈当成当前任务的输出条件，而不是背景解释。特别是在多约束场景中，结构化编码往往比纯自然语言更能减少漏改和重复犯错。³⁸

示例对比编码是我看好的单一增强手段。它不是只告诉模型“错了”，而是提供一个极小的“偏好学习”现场：`bad action -> 为什么错` 对应 `good action -> 为什么更好`。Contrastive ICL 明确把 positive/negative example 对作为描述用户意图的方法；contrastive prompting 则表明，让模型显式对照正误候选，本身就能提升复杂推理。对 planner 尤其如此，因为 action correction 通常不是开放式写作，而是在**少数可行后继动作之间重排序**。²²

逻辑规则或原则适合“硬约束明显”的问题，例如安全规则、格式规范、工具调用前提、状态机约束。Constitutional AI 说明自然语言原则本身可以成为高层控制信号；IFEval 和 DeCRIM 则说明可验证约束利于评测与修正。对你的设定，这意味着若 verifier 已经能产出“违反了哪条规则/前提”，应当把它作为专门字段，而不只是埋在长解释里。³⁹

置信度分数最不应被单独使用。Kadavath 等表明模型对“我知道/不知道”有一定识别能力，但 Xiong、Yang 与 Li 等都显示 verbalized confidence 既容易过度自信，也强烈依赖 prompt 形式。因此，建议把置信度用于**策略门控**而非内容劝说：例如高置信错误触发“禁止重复原 action + 必须给出替代 action”；中置信错误触发“局部解释后重写”；低置信错误则只要求“列出两个候选 action 并标明证据不足”。⁴

反事实情景值得覆盖，但我会把它放在“有针对性使用”而不是默认模板。其最佳用途是把 verifier 的理由写成一条最小状态变动：“如果继续执行当前 action，会导致约束 c 被违反；如果改为 action a' ，则 c 与 d 同时满足。”这类编码对计划修补、时序理解、前提-效果链特别有用；但现阶段更强的实验支持仍主要来自特定 counterfactual consistency 任务，而不是普适 agent planning。⁴⁰

综合上面几类证据，我的建议排序是：**混合编码（自然语言 + 结构化标签）**最稳，其次是**对比式编码**，再其次是**逻辑规则**，然后才是**纯自然语言**；**纯置信度分数**与**角色设定**都不应作为主力机制。更具体地说，如果你要写一条默认 prompt，我建议最小结构至少包含以下槽位：`原 action`、`错误点`、`违反的约束/前提`、`自然语言理由`、`禁止重复原 action`、`要求给出一个替代 action`。这个模板之所以强，不是因为它更长，而是因为它同时利用了过程监督、结构化约束与对比性三种机制。⁴¹

还有一个常被忽略但实际影响很大的问题，是**理由放在哪里**。Lost in the Middle 与顺序敏感性研究都表明，相关信息埋在长上下文中部时效果会明显下降；因此 verifier 理由最好放在**距离 action 生成最近的位置**，而不是作为对话历史很早的一段。若历史很长，优先做“摘要后重贴近 action”而不是一次性全量拼接。⁴²

评估方法与实验设计建议

这个问题非常适合做**可重复、因果清晰的对照实验**，因为 verifier 已经外生地给出了“错误点与理由”。Kamoi 等特别强调，自我纠错研究中常见的问题是研究问题定义含糊、比较不公平、以及把“外部帮助”与“模型自我改正”混在一起。因此，实验设计首先要控制变量固定：同一 base model、同一 verifier、同一 decoding

参数、同一 token 预算、同一最大修正轮数、同一任务实例集合；变化的只应是**理由如何编码与提示如何调度**。⁴³

任务类型上，我建议至少覆盖三类。第一类是**交互式 action 规划**，如 ALFWorld 与 WebShop，这两类环境天然包含离散 action、状态转移与可验证任务成功率；第二类是**可自动验证的约束跟随**，如 IFEval 与 ReallInstruct，适合精确测“违反了哪条约束后，prompt 是否能把输出拉回正确区域”；第三类是**程序/工具 action 修补**，如 HumanEval 或代码补丁任务，便于用单元测试验证修正是否真正确。这样可以分别考察 action repair、constraint repair 与 executable repair，而不是把它们混成单一指标。⁴⁴

指标设计上，除你已经点名的成功率、收敛速度、循环次数、信息效率、鲁棒性与对抗性外，我建议再加入两个对该问题非常关键的量。其一是**首轮修正成功率**，即 verifier 第一次指出问题后，模型是否能在一次重写内改成正确 action；其二是**过度修正率**，即原本可行或边界可行的 action，是否因 prompt 过强而被错误推翻。信息效率可以定义为每增加 100 个提示 token 带来的成功率提升；鲁棒性可以分解为**理由 paraphrase、字段顺序、位置、噪声比例、错误标签粒度**五个维度。对 agent 任务，还应同时报告**任务级成功率与步骤级修正精度**，否则容易把“后面又撞对了”误判成该次 intervention 有效。⁴⁵

统计比较建议使用**配对**而不是非配对方案，因为所有提示条件都作用于同一批实例。对二元成功/失败指标，可用 McNemar 检验；对成功率差、token 成本差、循环次数差这类均值或差值指标，可报告 paired bootstrap 置信区间。Dietterich 关于分类器比较的经典结论仍适用：比较同一测试集上两个方法的成败分布时，McNemar 很合适；而 Bestgen 进一步建议在 NLP 中重视 bootstrap 置信区间，而不是只报“是否显著”。

⁴⁶

消融研究应当直接检验“哪一部分真正促成了行为改变”。最关键的消融不是模型大小，而是：去掉替代 action 槽位、去掉结构化标签、把 verifier 理由移到长上下文中部、把对比式编码换成纯自然语言、把高置信错误的硬约束改成软建议、把单次修正改成多轮反思。这样，你能分辨提升究竟来自**内容本身**，还是来自**位置、格式、对比性与控制强度**。这正是该问题最适合做论文的地方。⁴⁷

下表给出三组我认为最适合学术论文的实验假设。它们都尽量避免多层工程设计，而把问题处理成“同一 verifier、不同 prompt 干预”的干净对照。⁴⁸

假设	方法或提示模板	数据或任务类型	评价指标	预期结果
混合理由编码优于单一编码	比较三种输入： 仅自然语言理由、仅结构化标签、自然语言+结构化标签； 统一模板都包含原 action，但仅混合版额外包含 forbid_repeat 与 constraint list。该设计直接对应 DeCRIM 的约束分解思想与 ICL 的 format hypothesis。 ⁷	IFEval、ReallInstruct，以及由 ALFWorld/ WebShop 轨迹扰动构造的 action-repair 集。 ⁴⁹	指令级成功率、约束级成功率、首轮修正成功率、McNemar、paired bootstrap CI。	混合编码在多约束任务与离散 action 任务上显著优于两种单一编码，且 token 增量相对可控。
对比式修正优于泛化反思	比较 请反思并重写 与 bad action / good action 对比式提示；对比版要求先用一句话说明原 action 为何错，再直接给一个替代 action，不允许重复旧 action。该假设来自 contrastive ICL、contrastive prompting 与外部 critique 文献。 ⁵⁰	ALFWorld、WebShop、HumanEval 或代码补丁集。 ⁵¹	成功率、首轮修正成功率、平均循环次数、token/成功增量、过度修正率。	对比式版本应在同等或更低轮数下得到更高修正率，尤其对“已知一个明显坏 action”的实例最明显。

假设	方法或提示模板	数据或任务类型	评价指标	预期结果
置信度门控的单次回溯优于固定轮数反思	用 verifier 置信度或严重性分数做门控：高置信错误执行硬禁止+局部替代，中置信错误执行解释后替代，低置信错误执行列两候选并说明不确定性；所有条件最多一轮局部修正，必要时只允许回溯一步。该设计受 confidence-aware self-correction 与外部反馈综述启发。 ⁵²	WebShop/ALFWorld 长轨迹子集，外加多轮代码修补任务。 ⁵³	任务成功率、平均轮数、死循环率、token 成本、鲁棒性、校准误差。	相比“总是反思”或“总是回溯”，门控策略应提高信息效率，并显著减少循环与过度修正。

潜在风险与失败模式

最直接的失败模式是**循环与性能退化**。Huang 等与 Kamoi 等都指出，推理型任务中的 intrinsic self-correction 很容易在没有新信息的情形下越改越差；而一旦把“反思”做成默认多轮机制，错误很可能从“原 action 错”变成“在错误的 critique 上继续迭代”。因此，缓解策略应当是：**把修正轮数限制为一到两轮；把 verifier 理由视作必须引入的新证据；若连续两轮得到相同失败理由，则停止而不是继续“想更多”**。²⁵

第二类失败模式是**提示污染、上下文稀释与外部注入**。Lost in the Middle 说明长上下文本身就会稀释关键理由；prompt injection 文献则进一步表明，若把页面文本、工具返回、历史对话与 verifier 理由混在一起，模型很容易把“数据”误当成“指令”。因此，缓解方式不是一味增加 guardrail，而是**显式分区**：把 verifier 理由做成独立槽位与固定 schema，把环境文本与外部文档声明为“证据而非指令”，并尽量把真正驱动 action 的理由放在输出 action 之前的最近位置。⁵⁴

第三类失败模式是**过度依赖 verifier 与确认偏差**。如果模型已经倾向于迎合给定说法，那么“带理由的 verifier”在有噪声时反而可能让模型更坚信一个错误的修正方向。关于 sycophancy 的研究表明，RLHF 模型会系统性地偏向迎合用户或上下文中看似权威的立场，而不一定坚持客观正确性。对这个问题，最重要的缓解方式是**把 verifier 的 authority 改写成 evidence**：不是“因为 verifier 这么说所以你必须改”，而是“以下哪条前提未满足/哪条约束被违反，因此必须换 action”。换言之，应减少“服从某个声音”的表达，增加“基于证据重选”的表达。⁵⁵

第四类失败模式是**把角色/身份与置信度误当主信号**。角色设定在客观正确率问题上通常不稳定；不同 persona 还可能引入额外偏差。置信度同样如此：若只是让模型报一个很高或很低的数字，并不等于模型真的完成了 error localization。缓解策略是把 persona 限制在风格层，不让它承担纠错职责；把置信度限制为门控变量，并始终要求与具体错误点或约束绑定。⁵⁶

推荐的精简可发表贡献

如果目标是做一篇**聚焦明确、可验证、不过度工程化**的论文，我最推荐的贡献不是再造一个复杂 agent 架构，而是把问题缩成“**同一个 verifier，何种理由编码最能让已有 planner 改写 action**”。这类贡献与 Kamoi 等对自我纠错研究边界的批评是吻合的：问题定义清晰、变量可控、对照可复现。⁴³

贡献一：动作局部对比修正提示。

提出一个最小模板，只做一件事：把 verifier 给出的错因压缩成 错误 action—错误点—违反约束—替代 action 四元组，并把它贴到当前决策点前。与基线 binary reject、generic reflect-and-revise、纯天然语言 critique 比较。这一贡献的创新点不在系统规模，而在于把 contrastive prompting、constraint following 与 process-level critique 合并成一个单轮干预模板；预期在 ALFWorld/WebShop/RealInstruct 上会比 generic reflection 更稳定。⁵⁷

贡献二：理由编码的因果消融基准。

从成功轨迹中人为扰动一个 action，令 verifier 产出对应理由，再比较不同编码：自然语言、结构化、混合、对比、反事实。这样做的价值在于把“错误本身”固定住，只比较 prompt 表达方式，从而真正回答“哪种理由编码更易促成行为改变”。这比直接在开放任务上比较 end-to-end success 更有解释力，也更符合 ICL/format/order literature 的研究传统。 58

贡献三：置信度门控的单次回溯策略。

不是研究更复杂的多轮 agent，而是研究**何时应该只局部重写，何时应该回溯一步**。把 verifier 置信度与错误严重性作为门控条件，比较“总是反思”“总是回溯”“置信度门控单次回溯”。它的论文价值在于：一方面回应了 confidence-aware self-correction 和 calibration 文献；另一方面避免把 test-time compute 膨胀成无法公平比较的多轮搜索。若结果成立，你得到的是一个很清晰的学术结论：**在已知错因条件下，门控策略比盲目增加反思轮数更有效。** 52

总的来说，若只保留一句核心结论，我会写成：**在 verifier 已知错误点与理由的前提下，最有效的 prompt 干预不是泛化地“让模型更谨慎”，而是把错因压缩为贴近当前决策点的局部、结构化、对比式证据，使模型不得不在一个更窄、更可执行的 action 空间中重选。**这一路线既符合现有理论与实验，也最适合形成清晰、可发表的学术贡献。 59

1 5 9 10 43 45 48 59 <https://arxiv.org/abs/2406.01297>

<https://arxiv.org/abs/2406.01297>

2 7 26 27 35 36 <https://arxiv.org/abs/2410.06458>

<https://arxiv.org/abs/2410.06458>

3 14 19 42 47 54 <https://arxiv.org/abs/2307.03172>

<https://arxiv.org/abs/2307.03172>

4 <https://arxiv.org/abs/2207.05221>

<https://arxiv.org/abs/2207.05221>

6 41 <https://arxiv.org/abs/2305.20050>

<https://arxiv.org/abs/2305.20050>

8 13 22 50 57 <https://arxiv.org/abs/2401.17390>

<https://arxiv.org/abs/2401.17390>

11 <https://arxiv.org/abs/2203.02155>

<https://arxiv.org/abs/2203.02155>

12 38 58 <https://arxiv.org/abs/2202.12837>

<https://arxiv.org/abs/2202.12837>

15 A Systematic Survey of Prompting Techniques

https://arxiv.org/html/2406.06608v4?utm_source=chatgpt.com

16 <https://arxiv.org/abs/2205.11916>

<https://arxiv.org/abs/2205.11916>

17 25 <https://arxiv.org/abs/2310.01798>

<https://arxiv.org/abs/2310.01798>

18 <https://arxiv.org/abs/2110.08387>

<https://arxiv.org/abs/2110.08387>

- 20 40 <https://arxiv.org/abs/2502.11425>
<https://arxiv.org/abs/2502.11425>
- 21 <https://arxiv.org/html/2505.11839v1>
<https://arxiv.org/html/2505.11839v1>
- 23 <https://arxiv.org/abs/2101.06804>
<https://arxiv.org/abs/2101.06804>
- 24 37 Self-Refine: Iterative Refinement with Self-Feedback
https://arxiv.org/abs/2303.17651?utm_source=chatgpt.com
- 28 <https://arxiv.org/abs/2205.10625>
<https://arxiv.org/abs/2205.10625>
- 29 <https://arxiv.org/abs/2305.10601>
<https://arxiv.org/abs/2305.10601>
- 30 31 <https://arxiv.org/abs/2211.01910>
<https://arxiv.org/abs/2211.01910>
- 32 56 <https://aclanthology.org/2024.findings-emnlp.888/>
<https://aclanthology.org/2024.findings-emnlp.888/>
- 33 <https://arxiv.org/abs/2210.03629>
<https://arxiv.org/abs/2210.03629>
- 34 Reflexion: Language Agents with Verbal Reinforcement Learning
https://arxiv.org/abs/2303.11366?utm_source=chatgpt.com
- 39 <https://arxiv.org/abs/2212.08073>
<https://arxiv.org/abs/2212.08073>
- 44 51 53 <https://arxiv.org/abs/2010.03768>
<https://arxiv.org/abs/2010.03768>
- 46 <https://pubmed.ncbi.nlm.nih.gov/9744903/>
<https://pubmed.ncbi.nlm.nih.gov/9744903/>
- 49 <https://arxiv.org/abs/2311.07911>
<https://arxiv.org/abs/2311.07911>
- 52 <https://arxiv.org/abs/2402.12563>
<https://arxiv.org/abs/2402.12563>
- 55 <https://arxiv.org/abs/2310.13548>
<https://arxiv.org/abs/2310.13548>